Build Custom Slim Pandas Layer (No Docker)

Overview

Build a custom pandas Lambda layer (75 MB) to replace AWS managed layer (258 MB), unblocking data-transfer Lambda deployment.

Method: Local Python 3.12 + pip

PHASE 1: Build Custom Pandas Layer

TASK 1.1: Set Up Build Environment

```
mkdir -p ~/lambda-layers/pandas-python312/python/lib/python3.12/site-packages
cd ~/lambda-layers/pandas-python312
```

Verify: Directory structure created

TASK 1.2: Verify Python 3.12

```
python3.12 --version
```

Expected: Python 3.12.x

If not installed:

```
# Ubuntu/Debian
sudo apt update && sudo apt install -y python3.12 python3.12-venv python3.12-c

# Mac
```

```
brew install python@3.12

# Amazon Linux 2023
sudo yum install -y python3.12 python3.12-pip python3.12-devel
```

TASK 1.3: Create requirements.txt

File: `~/lambda-layers/pandas-python312/requirements.txt`

```
pandas==2.2.1
numpy==1.26.4
python-dateutil==2.9.0
pytz==2024.1
tzdata==2024.1
six==1.16.0
```

TASK 1.4: Install Packages

```
pip3.12 install \
  -r requirements.txt \
  -t python/lib/python3.12/site-packages/ \
  --platform manylinux2014_x86_64 \
  --implementation cp \
  --python-version 3.12 \
  --only-binary=:all: \
  --upgrade
```

Verify:

```
ls python/lib/python3.12/site-packages/
# Should show: pandas/, numpy/, dateutil/, pytz/, tzdata/, six.py
```

TASK 1.5: Clean Up (Reduce Size)

```
# Remove tests
find python -type d -name "tests" -exec rm -rf {} + 2>/dev/null || true
find python -type d -name "__pycache__" -exec rm -rf {} + 2>/dev/null || true

# Remove bytecode
find python -name "*.pyc" -delete
find python -name "*.pyo" -delete

# Remove metadata
find python -name "*.dist-info" -exec rm -rf {} + 2>/dev/null || true
find python -name "*.egg-info" -exec rm -rf {} + 2>/dev/null || true

# Remove numpy tests/docs
rm -rf python/lib/python3.12/site-packages/numpy/tests
rm -rf python/lib/python3.12/site-packages/numpy/doc

# Remove pandas tests
rm -rf python/lib/python3.12/site-packages/pandas/tests
```

Verify size:

```
du -sh python/
# Expected: 70-80 MB
```

TASK 1.6: Verify Package Structure

```
# Test imports
python3.12 -c "import sys; sys.path.insert(0, 'python/lib/python3.12/site-pack

python3.12 -c "import sys; sys.path.insert(0, 'python/lib/python3.12/site-pack
```

Expected:

```
Pandas: 2.2.1
Numpy: 1.26.4
```

TASK 1.7: Create ZIP

```
zip -r9 pandas-python312-layer.zip python
ls -lh pandas-python312-layer.zip
```

Expected size: 25-35 MB (compressed)

Verify contents:

```
unzip -l pandas-python312-layer.zip | head -20
```

TASK 1.8: Create Documentation

File: `~/lambda-layers/pandas-python312/LAYER_README.md`

```
# pandas-python312-slim Lambda Layer

**Version:** 1
**Created:** 2026-04-13
**Runtime:** Python 3.12
**Size:** 75 MB (uncompressed), 28 MB (compressed)

## Contents

| Package | Version |
|---------|---------|
| pandas | 2.2.1 |
| numpy | 1.26.4 |
| python-dateutil | 2.9.0 |
| pytz | 2024.1 |
| tzdata | 2024.1 |
| six | 1.16.0 |

## Layer ARN
```

arn:aws:lambda:eu-west-1:YOUR-ACCOUNT-ID:layer:pandas-python312-slim:1

```
(Update after upload)

## Comparison

| Metric | AWS Layer | Custom Layer | Savings |
|--------|-----------|--------------|---------|
| Size | 258 MB | 75 MB | 183 MB (71%) |
| Contents | pandas + boto3 + pyarrow + awswrangler | pandas + numpy only | Mi
| Cold Start | Slower | Faster | ~1 sec |
```

PHASE 2: Upload to AWS

TASK 2.1: Verify AWS CLI

```
aws --version
aws sts get-caller-identity
aws configure get region
```

If not configured:

```
aws configure
# Enter: Access Key, Secret Key, Region (eu-west-1), Format (json)
```

TASK 2.2: Upload Layer

```
cd ~/lambda-layers/pandas-python312

aws lambda publish-layer-version \
  --layer-name pandas-python312-slim \
  --description "Slim pandas layer for Python 3.12 - pandas 2.2.1 + numpy 1.26
  --license-info "BSD-3-Clause" \
  --compatible-runtimes python3.12 \
```

```
  --zip-file fileb://pandas-python312-layer.zip \
  --region eu-west-1
```

Save the LayerVersionArn from output:

```
echo "arn:aws:lambda:eu-west-1:123456789012:layer:pandas-python312-slim:1" > ]
```

TASK 2.3: Verify Upload

```
aws lambda list-layer-versions \
  --layer-name pandas-python312-slim \
  --region eu-west-1
```

PHASE 3: Test the Layer

TASK 3.1: Create Test Lambda

Directory: `~/lambda-test/pandas-layer-test`

File: `lambda_function.py`

```
import json
import pandas as pd
import numpy as np

def lambda_handler(event, context):
    results = {"status": "success", "tests": []}

    # Test 1: Versions
    try:
        results["tests"].append({
            "test": "Import versions",
            "status": "PASS",
            "pandas_version": pd.__version__,
```

```
        "numpy_version": np.__version__
    })
except Exception as e:
    results["tests"].append({"test": "Import versions", "status": "FAIL",

# Test 2: DataFrame
try:
    df = pd.DataFrame({'a': [1, 2, 3], 'b': [4, 5, 6]})
    results["tests"].append({
        "test": "Create DataFrame",
        "status": "PASS",
        "shape": list(df.shape)
    })
except Exception as e:
    results["tests"].append({"test": "Create DataFrame", "status": "FAIL",

# Test 3: Operations
try:
    total = df['a'].sum()
    results["tests"].append({
        "test": "Basic operations",
        "status": "PASS",
        "sum": int(total)
    })
except Exception as e:
    results["tests"].append({"test": "Basic operations", "status": "FAIL",

# Test 4: Numpy
try:
    arr = np.array([1, 2, 3])
    results["tests"].append({
        "test": "Numpy operations",
        "status": "PASS",
        "mean": float(np.mean(arr))
    })
except Exception as e:
    results["tests"].append({"test": "Numpy operations", "status": "FAIL",

passed = sum(1 for t in results["tests"] if t["status"] == "PASS")
results["summary"] = f"{passed}/{len(results['tests'])} tests passed"

return {
    'statusCode': 200 if passed == len(results["tests"]) else 500,
```

```
        'body': json.dumps(results, indent=2)
    }
```

TASK 3.2: Package and Deploy Test Lambda

```
cd ~/lambda-test/pandas-layer-test
zip lambda-test.zip lambda_function.py

# Get or create IAM role
aws iam list-roles --query 'Roles[?contains(RoleName, `lambda`)].{Name:RoleNam

# Deploy (replace ROLE_ARN and LAYER_ARN)
aws lambda create-function \
  --function-name pandas-layer-test \
  --runtime python3.12 \
  --role arn:aws:iam::ACCOUNT-ID:role/lambda-execution-role \
  --handler lambda_function.lambda_handler \
  --zip-file fileb://lambda-test.zip \
  --timeout 30 \
  --memory-size 512 \
  --layers $(cat ~/lambda-layers/pandas-python312/layer-arn.txt) \
  --region eu-west-1

# Wait for active
aws lambda wait function-active --function-name pandas-layer-test --region eu-
```

TASK 3.3: Test Invocation

```
aws lambda invoke \
  --function-name pandas-layer-test \
  --region eu-west-1 \
  response.json

cat response.json | jq .
```

Expected: All tests PASS

PHASE 4: Update data-transfer Lambda

TASK 4.1: Navigate to Repository

```
# Find repository
find ~ -maxdepth 3 -type d -name "*data-ingestion-pipeline*" 2>/dev/null

cd [path-to-repository]

# Verify
ls -la
# Should show: serverless.yml, lambdas/, .git/
```

TASK 4.2: Create Feature Branch

```
git checkout main
git pull origin main
git checkout -b fix/lambda-layer-size-pandas-slim
```

TASK 4.3: Backup Current Config

```
mkdir -p backups
cp serverless.yml backups/serverless.yml.backup-$(date +%Y%m%d-%H%M%S)
cp lambdas/mmm-data-transfer/requirements.txt backups/requirements.txt.backup-
```

TASK 4.4: Update serverless.yml

Find the data-transfer function (search for “mmm-data-transfer” or “AWSSDKPandas”)

Change FROM:

```
layers:
  - arn:aws:lambda:${self:provider.region}:464622532012:layer:Datadog-Extensic
  - arn:aws:lambda:${self:provider.region}:464622532012:layer:Datadog-Python31
  - arn:aws:lambda:${self:provider.region}:336392948345:layer:AWSSDKPandas-Pyt
```

Change TO:

```
layers:
  # Custom slim pandas layer (75 MB) replaces AWS layer (258 MB)
  - arn:aws:lambda:${self:provider.region}:YOUR-ACCOUNT-ID:layer:pandas-pythor
  # Datadog layers for monitoring
  - arn:aws:lambda:${self:provider.region}:464622532012:layer:Datadog-Extensic
  - arn:aws:lambda:${self:provider.region}:464622532012:layer:Datadog-Python31
```

Replace YOUR-ACCOUNT-ID:

```
aws sts get-caller-identity --query Account --output text
```

TASK 4.5: Update requirements.txt

File: `lambdas/mmm-data-transfer/requirements.txt`

Replace entire content with:

```
# All dependencies provided by layers or Lambda runtime:
#
# - pandas, numpy: pandas-python312-slim layer
# - boto3, botocore: Lambda Python 3.12 runtime (pre-installed)
# - python-dateutil: Transitive dependency of pandas
# - Datadog: Datadog layers
#
# DO NOT add packages here unless they are:
# 1. Not in Lambda runtime
# 2. Not in any attached layer
# 3. Actually imported by lambda_function.py
```

```
#
# Adding runtime-provided packages causes deployment to exceed 250 MB limit.
```

TASK 4.6: Update CI/CD (Optional)

File: `.github/workflows/deploy-sls-lambda.yaml`

Find pip install step, change FROM:

```yaml
- name: Install dependencies
  run: |
    for subdir in lambdas/*/; do
      if [ -f "${subdir}/requirements.txt" ]; then
        echo "Installing dependencies for ${subdir}"
        pip install -r "${subdir}/requirements.txt" -t "${subdir}" --quiet
      fi
    done
```

Change TO:

```yaml
- name: Install dependencies
  run: |
    for subdir in lambdas/*/; do
      if [ -f "${subdir}/requirements.txt" ]; then
        # Skip if requirements.txt is empty or only has comments
        if grep -qvE '^\s*(#|$)' "${subdir}/requirements.txt"; then
          echo "Installing dependencies for ${subdir}"
          pip install -r "${subdir}/requirements.txt" -t "${subdir}" --quiet
        else
          echo "Skipping ${subdir} - no dependencies to install"
        fi
      fi
    done
```

TASK 4.7: Commit Changes

```
git add serverless.yml
git add lambdas/mmm-data-transfer/requirements.txt
git add .github/workflows/deploy-sls-lambda.yaml

git commit -m "Replace AWS Pandas layer with custom slim layer

- Replace AWSSDKPandas-Python312:1 (258 MB) with pandas-python312-slim:1 (75 M
- Remove duplicate boto3/botocore from requirements.txt (saves 80-90 MB)
- Keep Datadog layers for monitoring (40 MB)
- Update CI/CD to skip empty requirements.txt

Size breakdown:
- Before: 380 MB (130 MB over limit) ❌
- After: 115.5 MB (134.5 MB under limit) ✅

Fixes deployment blocker: Lambda size exceeded 250 MB limit

Testing:
- Tested in pandas-layer-test function - all tests pass
- Verified imports work correctly
- Confirmed size under limit
"
```

TASK 4.8: Push Branch

```
git push origin fix/lambda-layer-size-pandas-slim
```

PHASE 5: Deploy and Validate

TASK 5.1: Deploy to Dev

```
serverless deploy --stage dev --verbose
```

Expected: Deployment succeeds (no size errors)

TASK 5.2: Verify Configuration

```
aws lambda get-function-configuration \
  --function-name sls-data-ingestion-pipeline-mmm-data-transfer-dev \
  --region eu-west-1 \
  --query '{Layers: Layers[].Arn, CodeSize, State}'
```

Verify:

- ✅ pandas-python312-slim layer present
- ✅ Both Datadog layers present
- ✅ NO AWSSDKPandas layer
- ✅ State: Active

TASK 5.3: Test Function

```
# Create test event
cat > test-event.json << 'EOF'
{
  "test": true,
  "retailer_id": "test-retailer"
}
EOF

# Invoke
aws lambda invoke \
  --function-name sls-data-ingestion-pipeline-mmm-data-transfer-dev \
  --payload file://test-event.json \
  --region eu-west-1 \
  response.json

cat response.json
```

Check CloudWatch Logs:

```
aws logs tail /aws/lambda/sls-data-ingestion-pipeline-mmm-data-transfer-dev --
```

Look for:

- ✅ No ImportError
- ✅ No ModuleNotFoundError
- ✅ Function executes successfully

TASK 5.4: Deploy to QA

```
git checkout main
git merge fix/lambda-layer-size-pandas-slim
git push origin main

serverless deploy --stage qa --verbose
```

Expected: QA deployment succeeds (was blocked before!)

TASK 5.5: Run Integration Tests

```
# Trigger Step Functions execution
aws stepfunctions start-execution \
  --state-machine-arn arn:aws:states:eu-west-1:ACCOUNT-ID:stateMachine:data-ir
  --input file://test-input.json \
  --region eu-west-1
```

Verify: Full pipeline runs successfully

PHASE 6: Cleanup

TASK 6.1: Delete Test Resources

```
# Delete test Lambda
aws lambda delete-function --function-name pandas-layer-test --region eu-west-

# Clean up local test files
rm -rf ~/lambda-test/pandas-layer-test
```

TASK 6.2: Document Layer

Create: `docs/LAMBDA_LAYERS.md` in repository

```
# Lambda Layers

## pandas-python312-slim

**ARN:** `arn:aws:lambda:eu-west-1:ACCOUNT-ID:layer:pandas-python312-slim:1`
**Size:** 75 MB
**Contents:** pandas 2.2.1, numpy 1.26.4
**Used by:** mmm-data-transfer Lambda

**Maintenance:**
- Owner: Platform Team
- Update: Quarterly
- Next review: 2026-07-13

**Rebuild instructions:** See `~/lambda-layers/pandas-python312/BUILD_INSTRUCT
```

Success Criteria

- ✅ Custom pandas layer built (75 MB)
- ✅ Layer uploaded to AWS
- ✅ Layer tested (all tests pass)
- ✅ serverless.yml updated
- ✅ requirements.txt cleaned
- ✅ Deployed to dev successfully

- ✅ Deployed to QA successfully (was blocked)
- ✅ Total size: ~115.5 MB (under 250 MB limit)
- ✅ Datadog monitoring retained
- ✅ Documentation updated

Troubleshooting

Issue: "No matching distribution found"
Fix: Check internet connection, verify package versions on PyPI

Issue: Package size >90 MB
Fix: Re-run cleanup commands

Issue: ImportError in Lambda
Fix: Verify layer ARN correct, check layer structure

Issue: Still over 250 MB
Fix: Verify AWS Pandas layer removed from serverless.yml

Issue: Deployment fails
Fix: Check AWS credentials, verify serverless.yml syntax